



CS 431/631 Introduction to Big Data

Overview of Machine Learning / Deep Learning

Some slides are borrowed from Yiran Chen, Hai Li, et. al.

Lei Yang
Computer Science & Engineering
University of Nevada, Reno



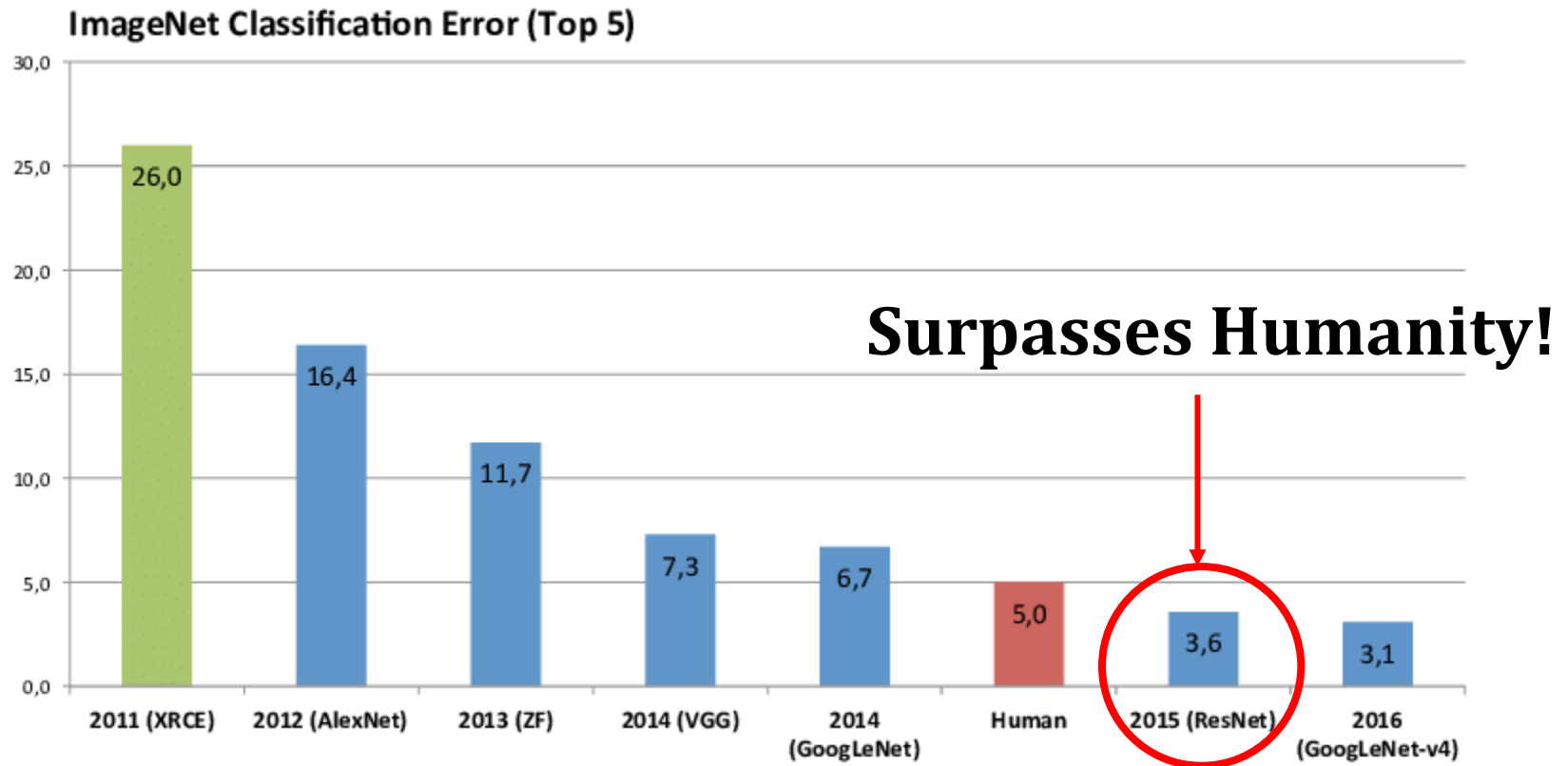
Outline

- Interesting applications of deep learning (DL)
- Categories of DL
- Important metrics of DL
- Platforms & Frameworks of DL

Applications

- Images
- Videos
- Speech Recognition
- Game; Strategy
- ... (many others)

Applications: Images



Can you tell
what kind of
turtle this is?



- A. *Dermochelys coriacea*
- B. *Caretta caretta*
- C. *Lepidochelys kempii*
- D. *Lepidochelys olivacea*

Applications: Images

- However, surprisingly weak...



Panda

57.7% confidence

+ 0.007 ×



noise

=



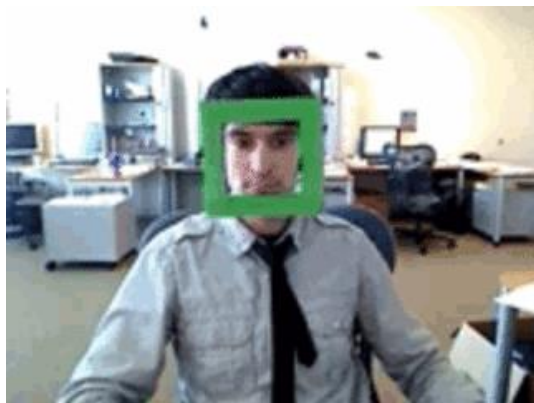
gibbon

99.3% confidence

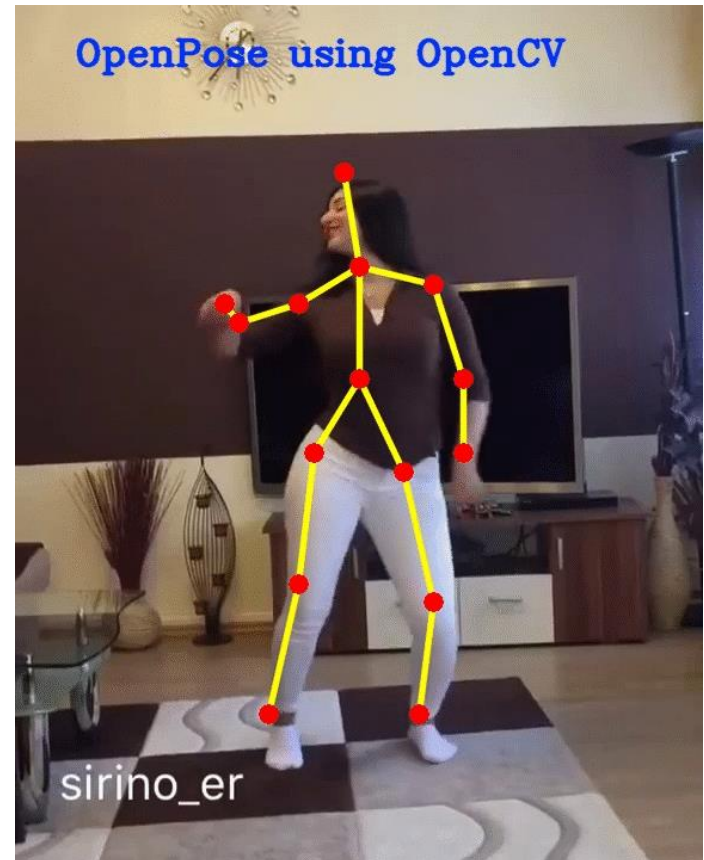
Applications: Videos



Object Detection

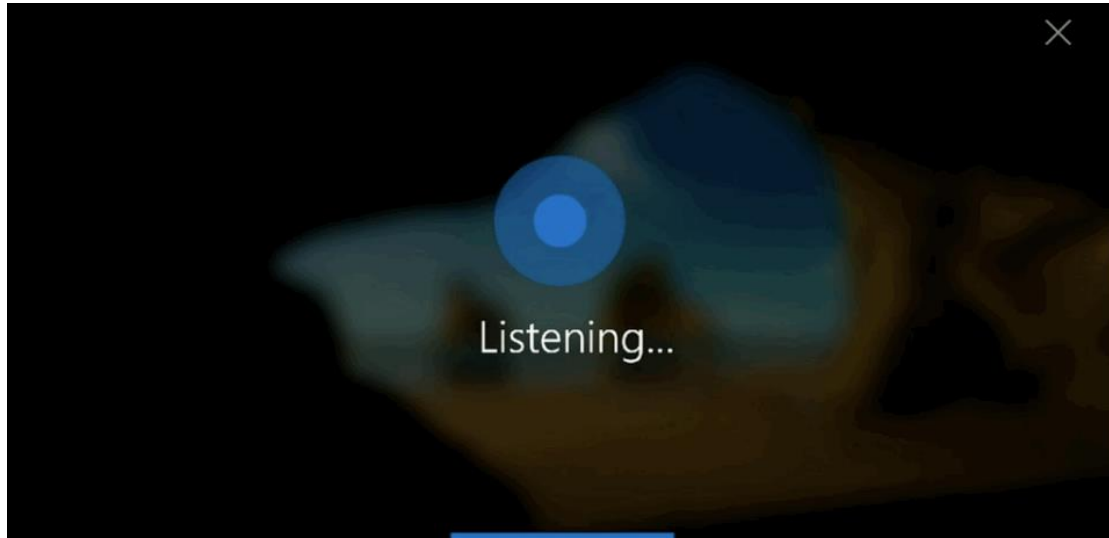


Real Time Face Tracking (??)

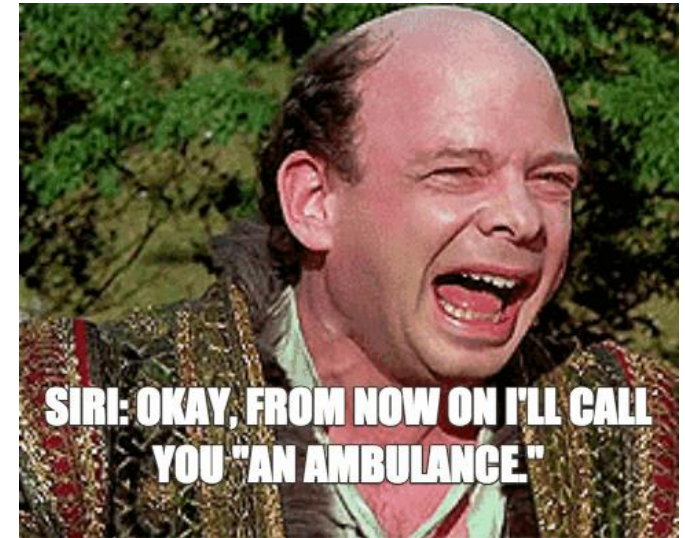


Human Pose Estimation

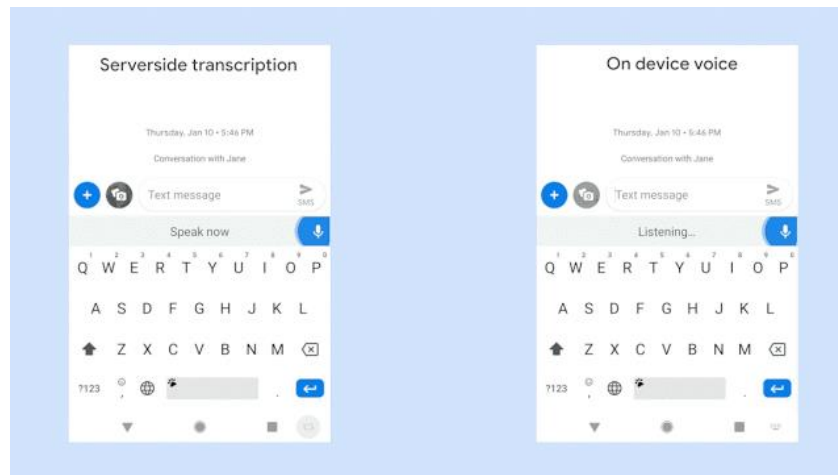
Applications: Speech



Cortana



"Siri, Call me an ambulance"

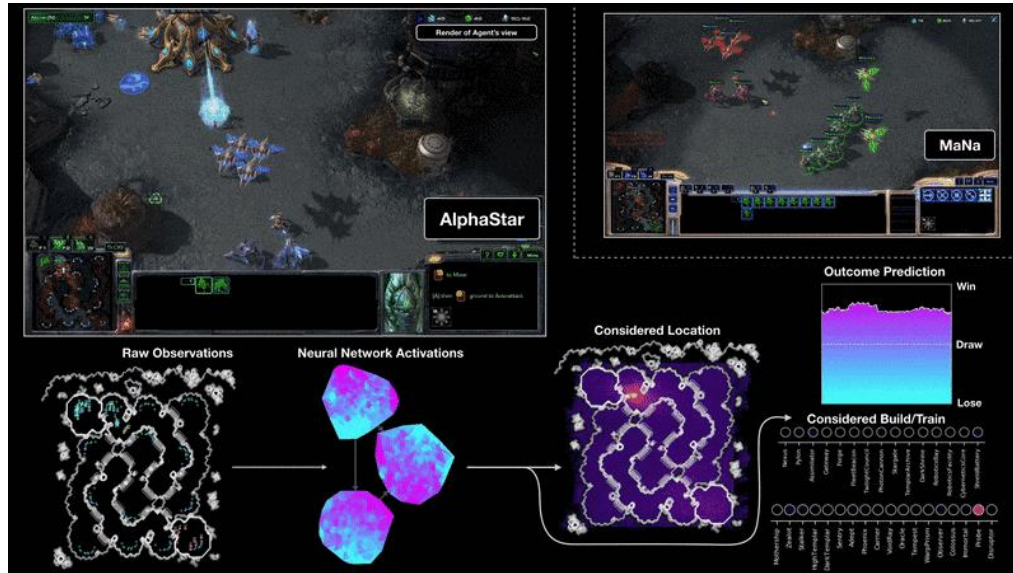


Speech To Text



Text To Speech

Applications: Game; Strategy



AlphaStar: StarCraft II

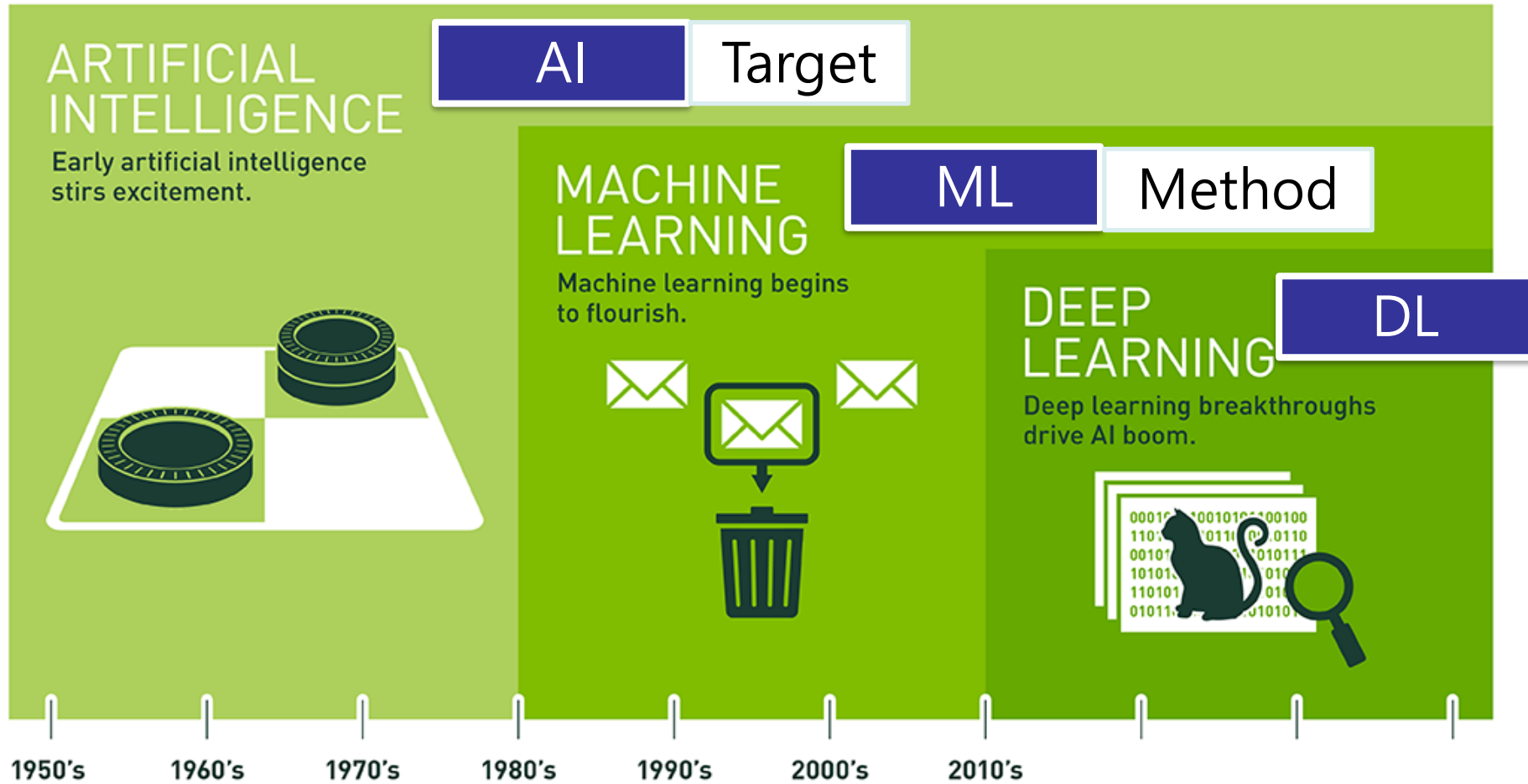


Alpha Go

Categories

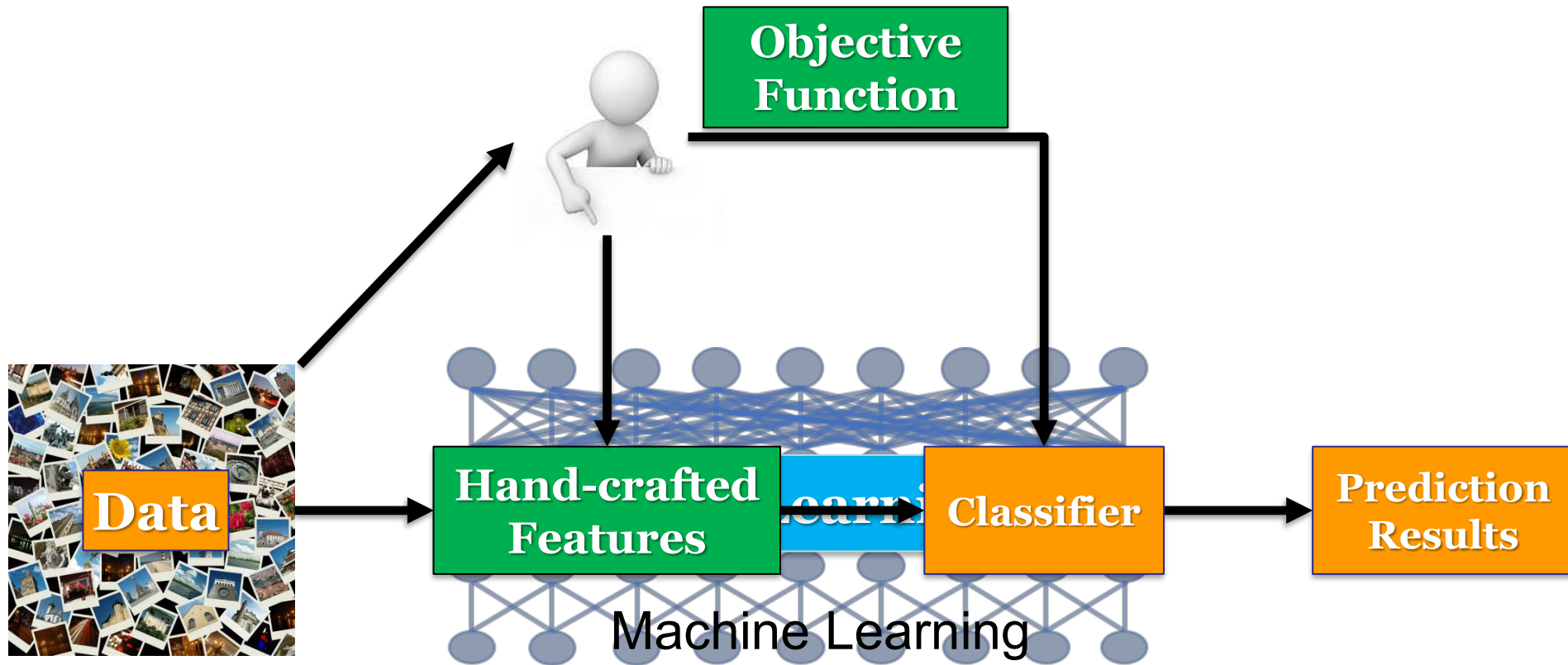
- Artificial Intelligence (AI) $\leftrightarrow$ Machine Learning (ML) $\leftrightarrow$ Deep Learning (DL)
- Historical View
- Structures of Neural Networks (NN)
- Learning types of NN

AI <-> ML <-> DL



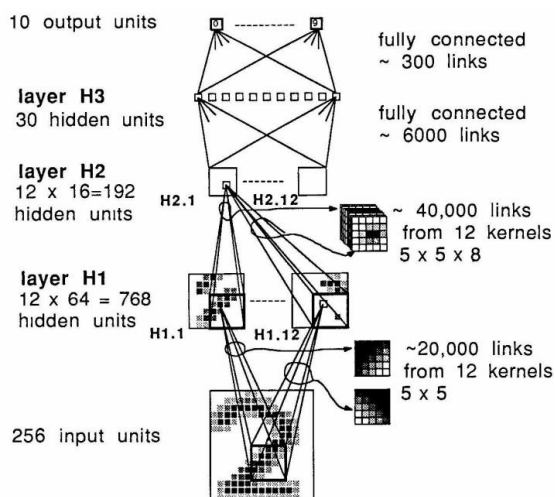
Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

ML vs DL



Historical View (Overview)

Convolutional Network (1980s)

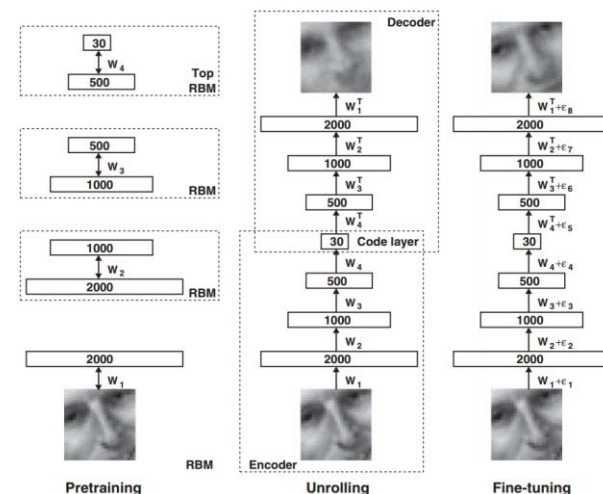


Dark period (1990s)

- Serious problem:
Vanishing gradient
- No benefits observed
by adding more layers
- No high performance
computing devices



Renaissance (2006 ~ Present)

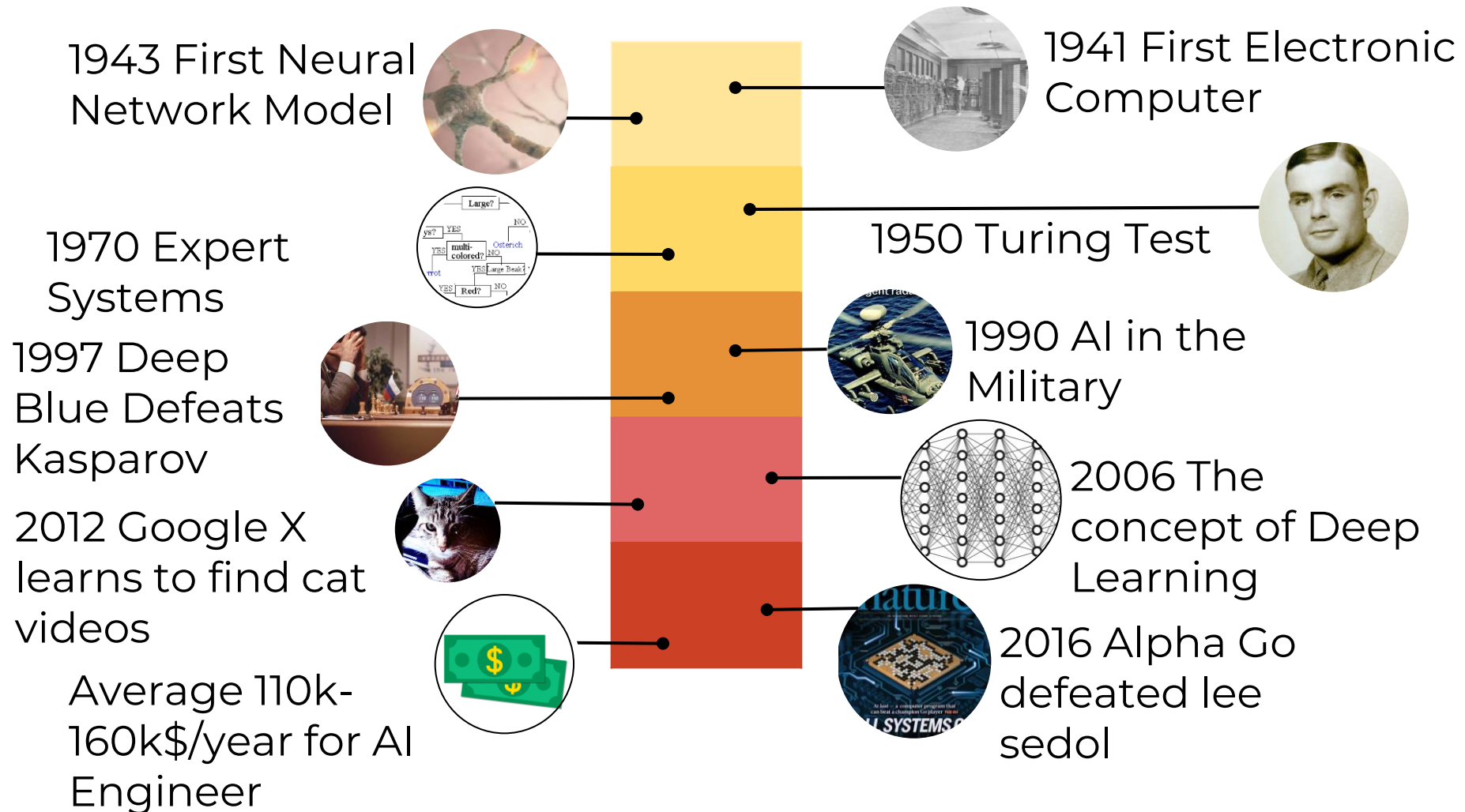


Y. Lecun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard and L. D. Jackel. *Backpropagation Applied to Handwritten Zip Code Recognition*. 1989.

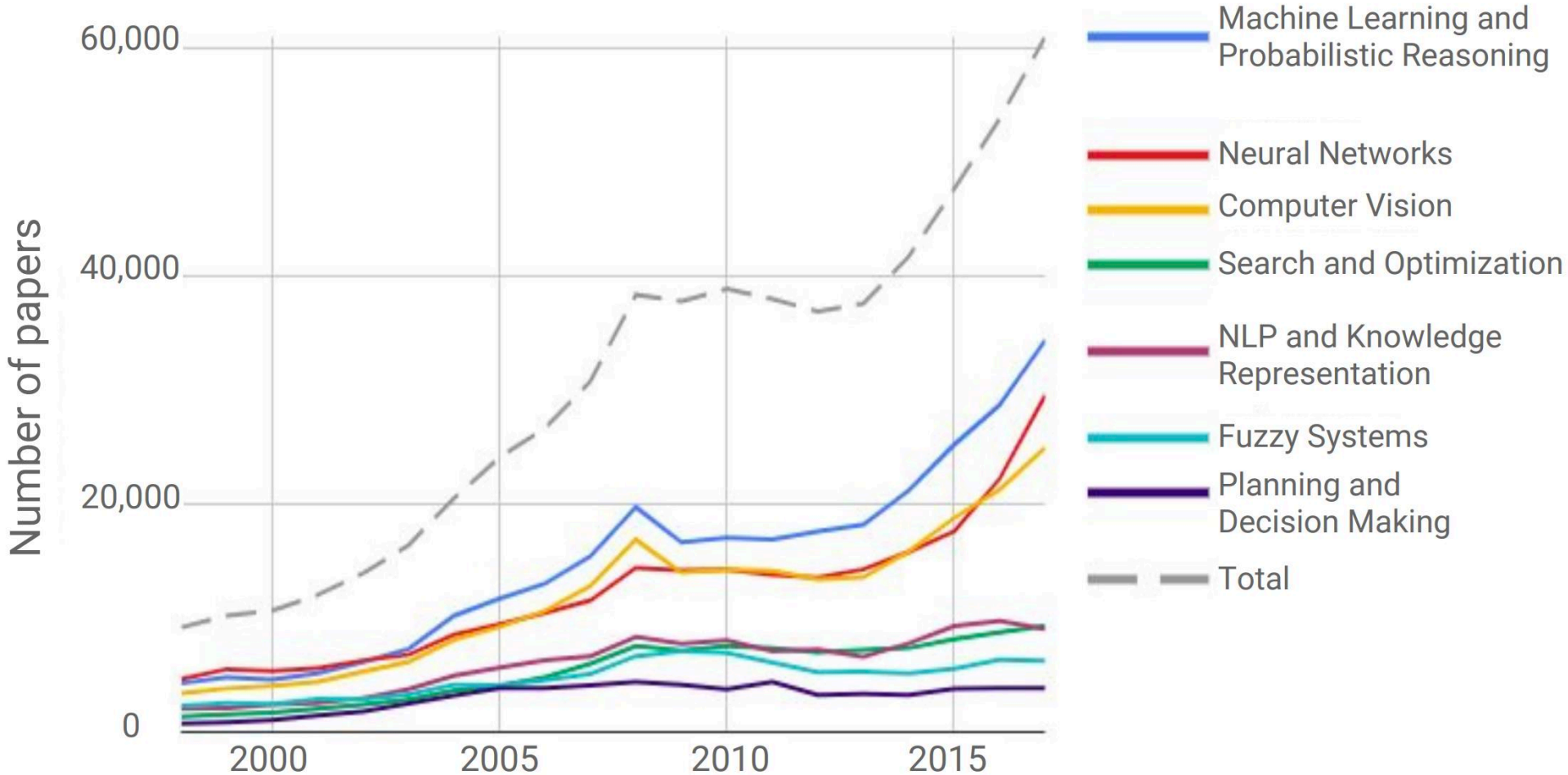
J. Schmidhuber. *Deep Learning in Neural Networks: An Overview*. arxiv, 2014.

G. E. Hinton and R. R. Salakhutdinov. *Reducing the Dimensionality of Data with Neural Networks*. Science, 2006.

Historical View (Milestones)

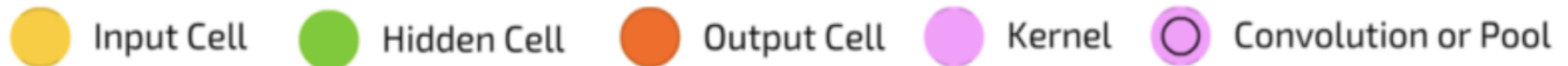
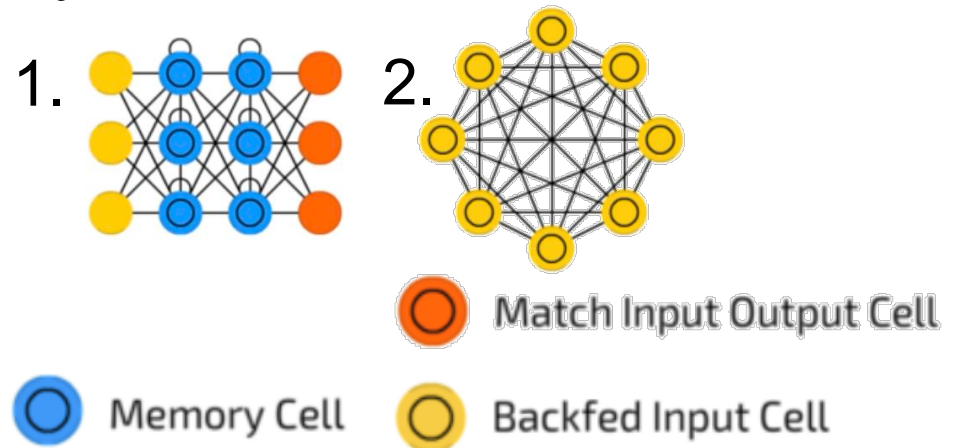
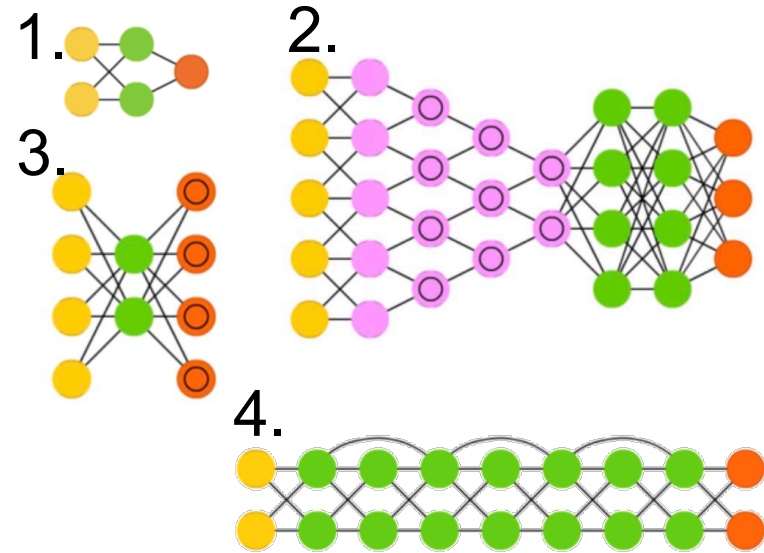


Historical View (Papers)



Structures

- Feedforward neural network:
 1. Multilayer perceptron
 2. Convolutional neural network
 3. Autoencoder
 4. Deep residual network
- Recurrent neural network:
 1. Long short-term memory
 2. Hopfield
 3. ...
- Spiking neural network

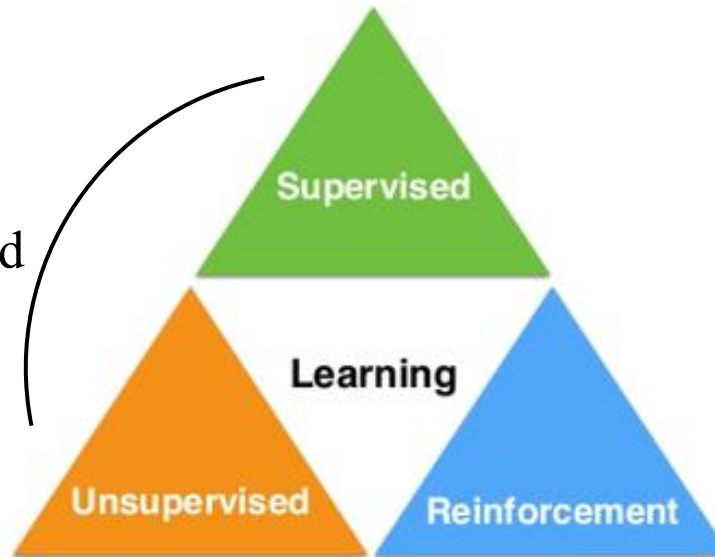


Learning types of NN

- Supervised
- Semi-supervised
- Unsupervised
- Reinforcement

Labeled data
Direct feedback
Predict outcome/future
Apps: Classification

Semi-supervised
Weakly-supervised



No labels
No feedback
Find hidden representations
Apps: Reconstruction

Decision process
Reward system
Learn series of actions
Apps: Decision-making

Important Metrics (LASER)

- Latency
- Accuracy
- Size of model
- Energy efficiency
- Robustness

Latency

- Latency is a measure of delay.
 - the **length of time** it takes for the data that you feed into one end of your network to emerge at the other end.
- Better Accuracy? Larger Latency!
- VGG-16 needs ~3s to process a single image on your smart phone , which is **Unacceptable**.

Going Deeper!

Deep = Many hidden layers

Error Rate 16.4%

AlexNet
(2012)

7.3%

VGG
(2014)

6.7%

GoogleNet
(2014)

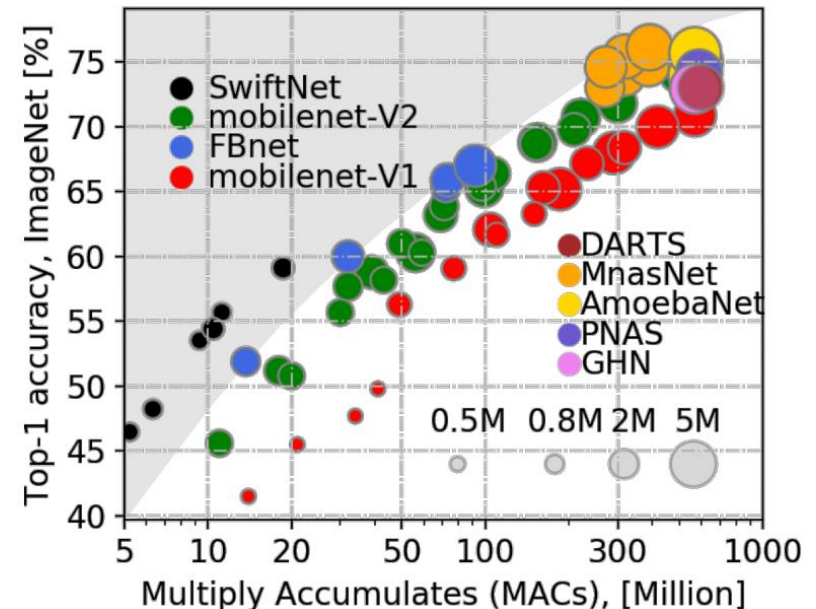
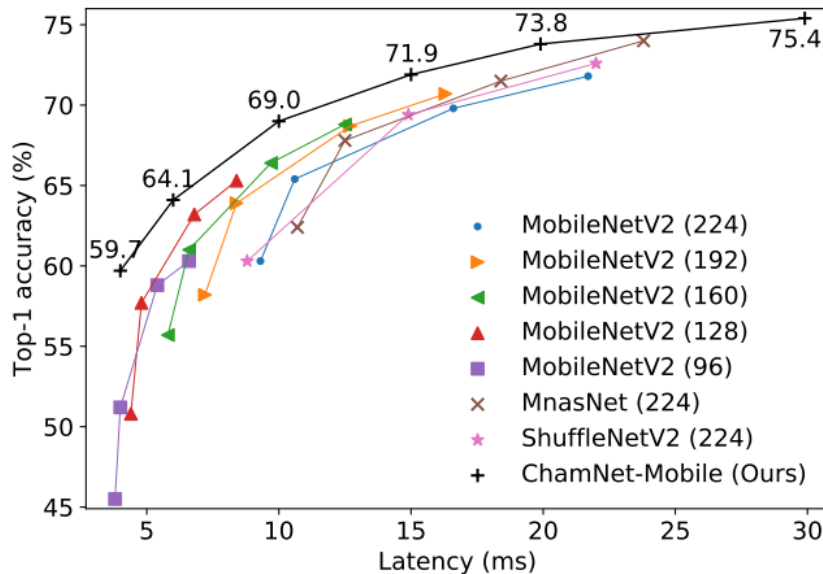
3.57%

152 layers

Residual Net
(2015)

Accuracy

- Accuracy is a metric for classification problem
- We call it: “Top K Accuracy”
- Accuracy is good, but we need to pay for it.
- Everything is a Trade-off.



Pareto Frontier

Size of model

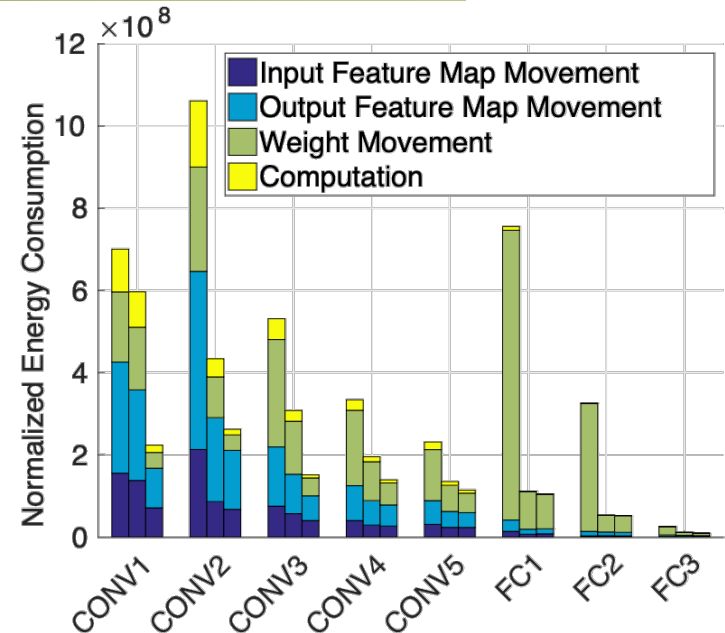
- #FLOP: Floating point operation per second.
- #MAC: **M**ultiply-**a**ccumulate operations
 - E.g., $\text{accumulator} = \text{accumulator} + (\text{input_a} * \text{input_b})$
- Usually, 1 floating-point Multiply-Accumulate is considered equivalent to 2 FLOPs.
- #Parameters

Energy efficiency

- Power Consumption [mW] [μ J]
 - 1 Watt = 1 Joule per second
- Energy is mainly used for
 - Calculation
 - Data movement

Energy is a different thing:

- A lower number of MACs **does not** necessarily lead to lower energy consumption.
- Convolutional layers **may consume more** energy than fully-connected layers.
- Deeper CNNs with fewer weights **do not** necessarily consume less energy than shallower CNNs with more weights.



Robustness

- Corresponds to the magnitude of the smallest bit of an 8 bit image encoding.

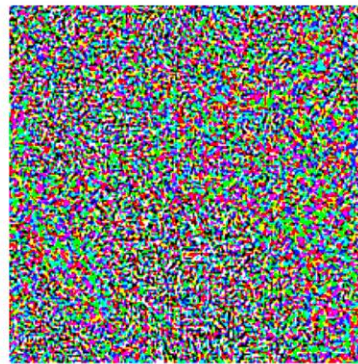


Panda

57.7% confidence

+

0.007 ×



noise

=



gibbon

99.3% confidence

- Usually, a high accuracy model is not robust.
- Compared to the size of a neural network, the **structure** has more impact towards robustness.

Everything is
a Trade-off

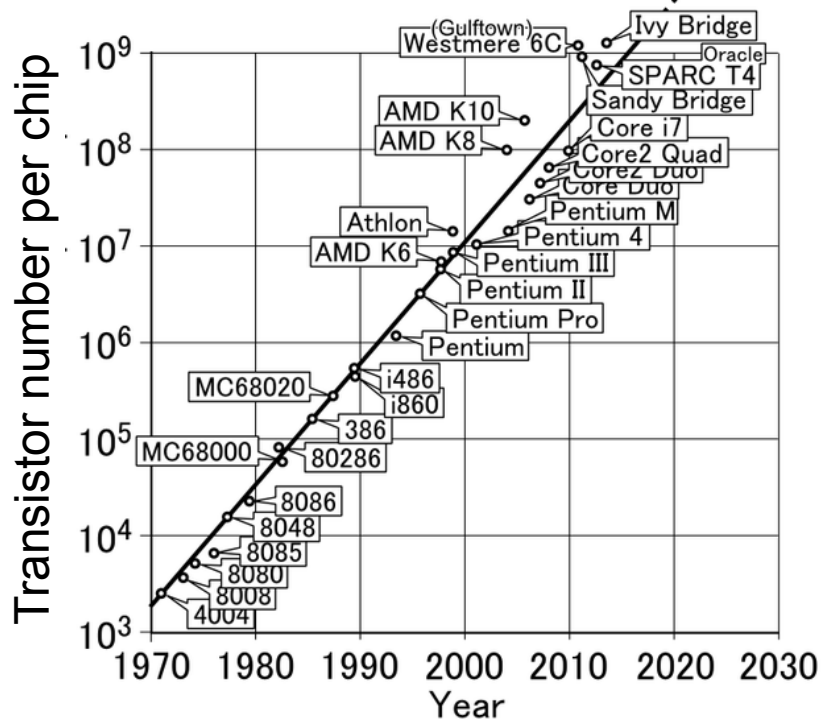
Platform & Frameworks

- **Hardware computing devices**
- **Software platform**

Hardware computing devices

Scale-up

Moore's Law



(Graph from betanews)

Scale-out



Parallel and Distributed

Hardware computing devices

Cost and speed are critical for both training and inference

GPU

- High power consumption
- Higher demand on data center cooling, power supply, and space utilization
- High speed

CPU

- Medium cost
- Medium power consumption
- Low speed

ASIC (Application-Specific Integrated Circuit)

- High non-recurring engineering
- Long design period, not suitable for fast iteration in deep learning development
- Very high speed

FPGA (Field Programmable Gate Arrays)

- Low power
- Low cost, hundreds of dollars
- Hardware reconfigurable
- Medium speed

ImageNet-1K Classification Performance

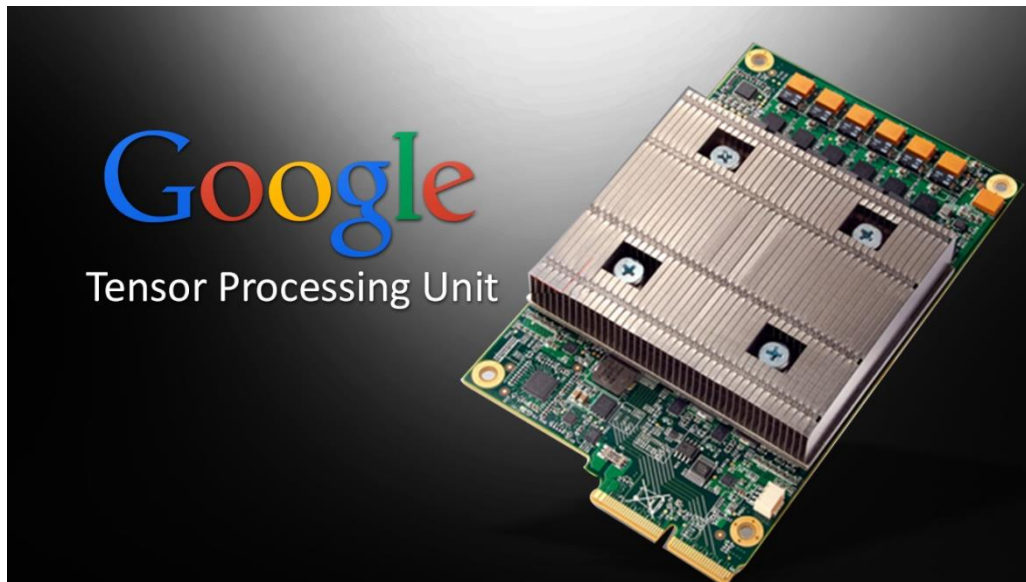
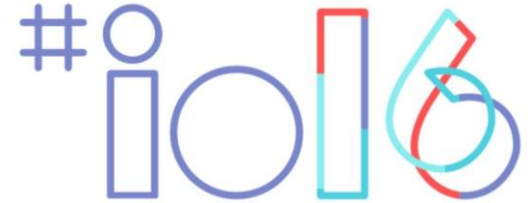
Platform	Inference Throughput	Peak TFLOPs	Effective TFLOPs	Power	Power Efficiency GOPs/J
Intel Xeon E5-2450	53 images/s	0.27T	0.074T (27%)	~225W	~0.3
Altera Arria 10 GX1150 (FPGA)	369 images/s	1.366T	0.51T (38%)	~40W	~12.8
NVIDIA Titan X	4129 images/s	6.1T	5.75T (94%)	~250W	~23.0

Neural network is usually trained in back-end GPU clusters, while FPGA is very suitable for low-power real-time inference job

Tensor Processing Unit (TPU)

Unveiled during Google I/O Conference, Mountain View, CA (May 2016).

Tensor Processing Unit (TPU): a custom ASIC built specifically for machine learning — and tailored for TensorFlow.



TPU

Applications:

1. RankBrain: improve the relevancy of search results.
2. Street View: improve the accuracy and quality of our maps and navigation.
3. AlphaGo: “think” much faster and look farther ahead between moves.



Server racks with TPUs used in the AlphaGo matches with Lee Sedol



Software Platform

 PyTorch

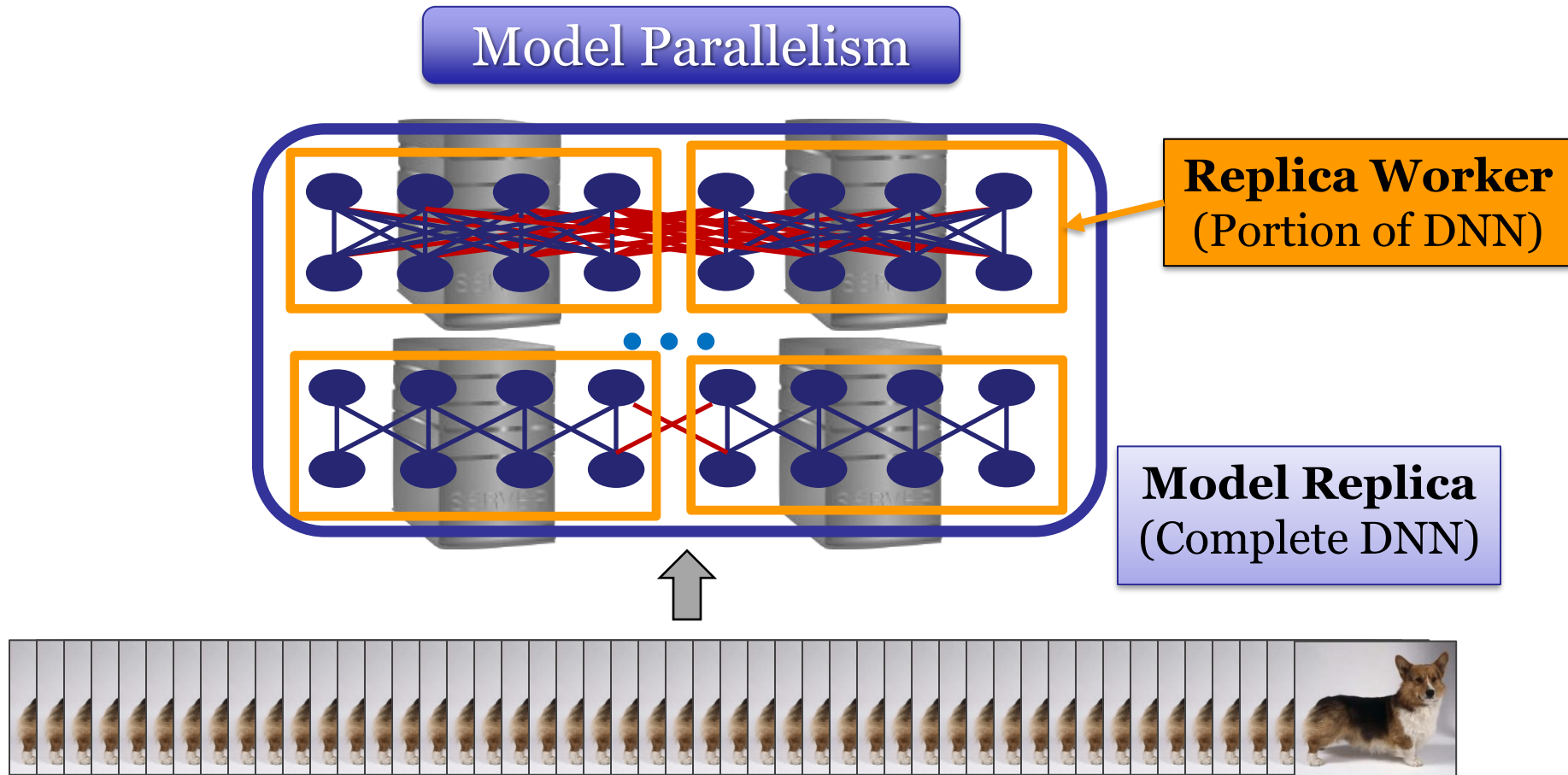

TensorFlow

 mxnet

 Microsoft
CNTK


Caffe2

Model Parallelism

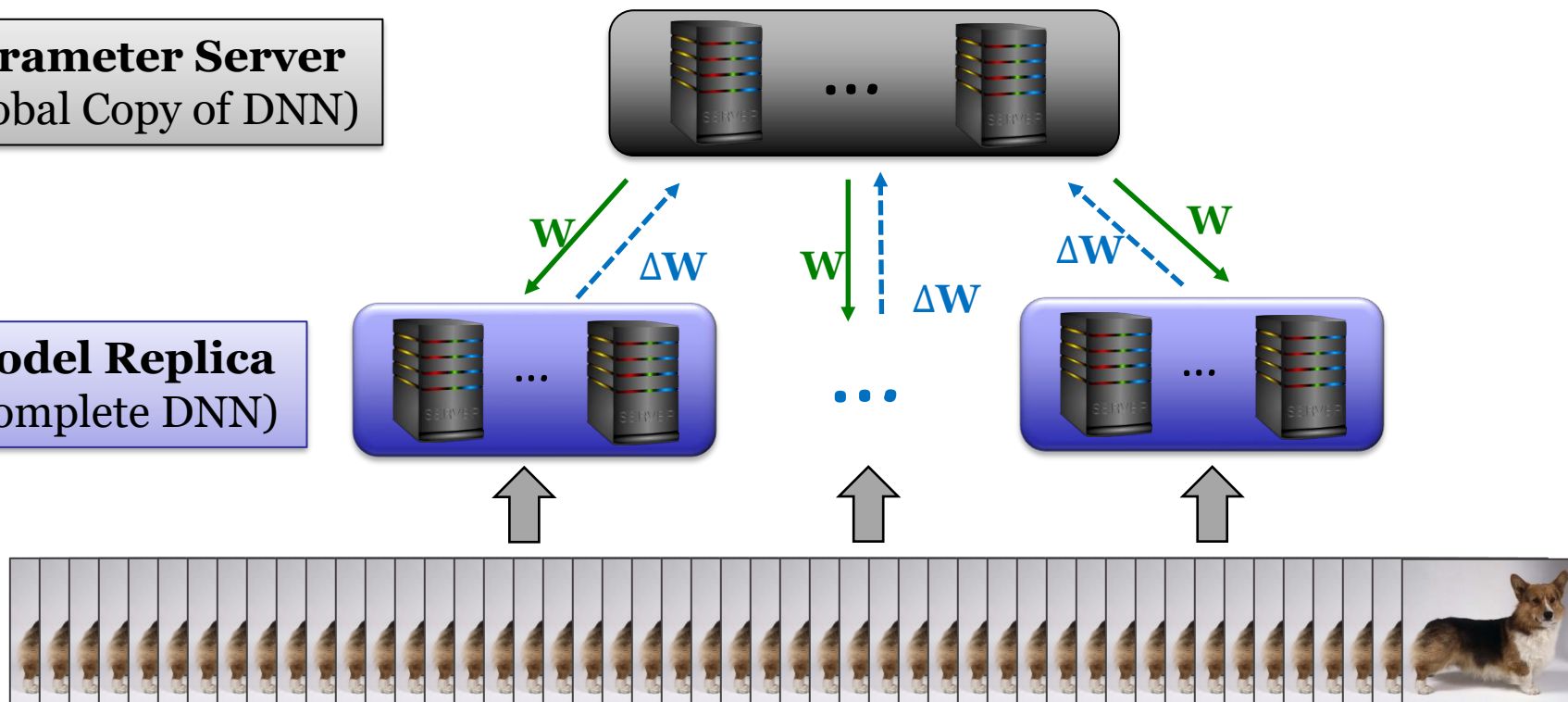


Data Parallelism

Data Parallelism

Parameter Server
(Global Copy of DNN)

Model Replica
(Complete DNN)



Comparison of Machine Learning Framework

	 PyTorch	 mxnet	 Chainer	 TensorFlow
Supported interfaces	C++, Python, Java, Rust, Go	C++, Python, Scala, Julia, Perl, MATLAB	C++, Python	C++, Python, Go, Java, Swift, JavaScript
Multi-GPU: Data parallel	Yes	Yes	Yes	Yes
Multi-GPU: Model parallel	Yes	Yes	Yes	Yes
Developed by	 facebook AI research group	 THE APACHE [®] SOFTWARE FOUNDATION	 Preferred Networks  NVIDIA [®]  intel  IBM	 Google Brain team

Tensorflow Pros/Cons

Pros

- **TensorBoard for visualization**
- **Data AND model parallelism; best of all frameworks**
- **Bigger developer community**

Cons

- Steep learning curve
- Usually slower than other frameworks right now
- Much “fatter” than Pytorch

Example TensorFlow operation types.

Category	Examples
Element-wise mathematical operations	Add, Sub, Mul, Div, Exp, Log, Greater, Less, Equal, ...
Array operations	Concat, Slice, Split, Constant, Rank, Shape, Shuffle, ...
Matrix operations	MatMul, MatrixInverse, MatrixDeterminant, ...
Stateful operations	Variable, Assign, AssignAdd, ...
Neural-net building blocks	SoftMax, Sigmoid, ReLU, Convolution2D, MaxPool, ...
Checkpointing operations	Save, Restore
Queue and synchronization operations	Enqueue, Dequeue, MutexAcquire, MutexRelease, ...
Control flow operations	Merge, Switch, Enter, Leave, NextIteration

Pytorch Pros/Cons

Pros

- **Dynamic approach to graph computation**
- **Usually faster than other DNN toolkits**
- **More convenient than Tensorflow**

Cons

- **Relatively smaller developer community compared to Tensorflow**
- **Less product oriented compared to Tensorflow or MXNet**

Pytorch vs. Tensorflow: MNIST

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(1, 20, 5, 1)
        self.conv2 = nn.Conv2d(20, 50, 5, 1)
        self.fc1 = nn.Linear(4*4*50, 500)
        self.fc2 = nn.Linear(500, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2, 2)
        x = x.view(-1, 4*4*50)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return F.log_softmax(x, dim=1)
```

<https://github.com/pytorch/examples/blob/master/mnist/main.py>

```
def cnn_model_fn(features, labels, mode):
    """Model function for CNN."""

    input_layer = tf.reshape(features["x"], [-1, 28, 28, 1])

    conv1 = tf.layers.conv2d(
        inputs=input_layer,
        filters=32,
        kernel_size=[5, 5],
        padding="same",
        activation=tf.nn.relu)

    pool1 = tf.layers.max_pooling2d(inputs=conv1, pool_size=[2, 2], strides=2)
    conv2 = tf.layers.conv2d(
        inputs=pool1,
        filters=64,
        kernel_size=[5, 5],
        padding="same",
        activation=tf.nn.relu)

    pool2 = tf.layers.max_pooling2d(inputs=conv2, pool_size=[2, 2], strides=2)
    pool2_flat = tf.reshape(pool2, [-1, 7 * 7 * 64])
    dense = tf.layers.dense(inputs=pool2_flat, units=1024, activation=tf.nn.relu)
    dropout = tf.layers.dropout(
        inputs=dense, rate=0.4, training=mode == tf.estimator.ModeKeys.TRAIN)

    logits = tf.layers.dense(inputs=dropout, units=10)

    predictions = {
        # Generate predictions (for PREDICT and EVAL mode)
        "classes": tf.argmax(input=logits, axis=1),
        # Add `softmax_tensor` to the graph. It is used for PREDICT and by the
        # `logging_hook`.
        "probabilities": tf.nn.softmax(logits, name="softmax_tensor")
    }
```

https://github.com/tensorflow/tensorflow/blob/master/tensorflow/examples/tutorials/layers/cnn_mnist.py

PyTorch in 5 Minutes



<https://www.youtube.com/watch?v=nbJ-2G2GXL0>

Tensorflow Tutorial for Python in 10 Minutes



https://www.youtube.com/watch?v=6_2hzRopPbQ&t=10s